Introduction to Data Structures and Algorithms

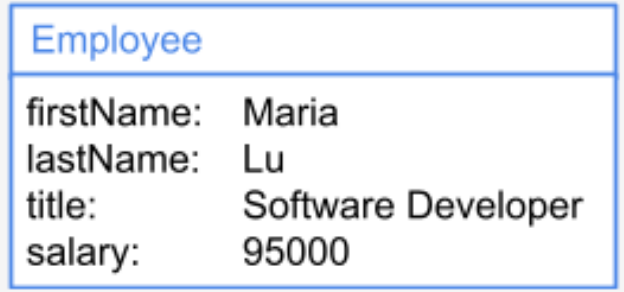
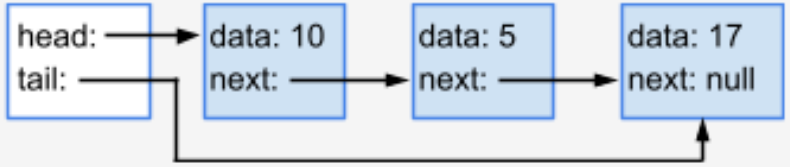
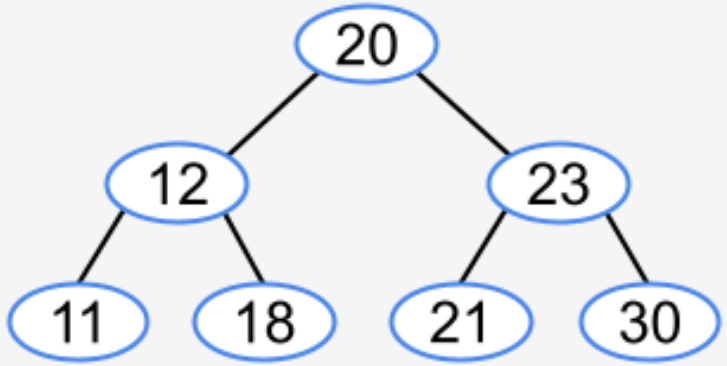
James Anderson

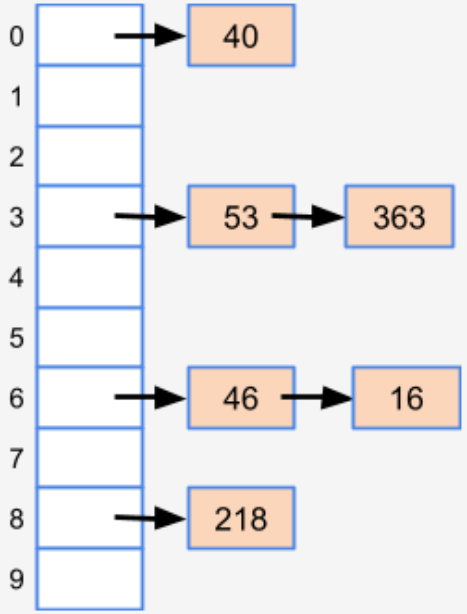
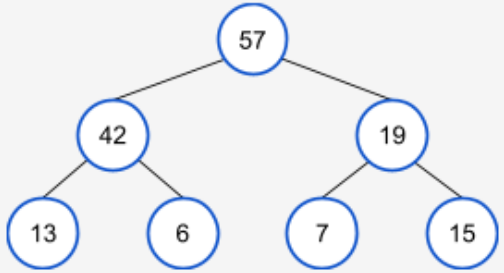
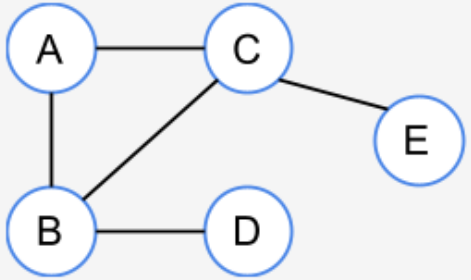
CS 3100/5100

Wright State University

Data Structures (DS)

- Way of organizing, storing, and performing operations on data
  - Accessing or updating
  - Searching for specific data
  - Inserting new data
  - Removing data
- How data is organized *in memory*

Data structure	Description	Example
Record	A ***record*** is the data structure that stores subitems, often called fields, with a name associated with each subitem.	
Array	An ***array*** is a data structure that stores an ordered collection of elements, where each element is directly accessible by a positional index.	
Linked list	A ***linked list*** is a data structure that stores an ordered list of items in nodes, where each node stores data and has a pointer to the next node.	
Binary tree	A ***binary tree*** is a data structure in which each node stores data and has up to two children, known as a left child and a right child.	

Hash table	A ***hash table*** is a data structure that stores unordered items by mapping (or hashing) each item to a location in an array.	
Heap	A ***max-heap*** is a tree that maintains the simple property that a node's key is greater than or equal to the node's childrens' keys. A ***min-heap*** is a tree that maintains the simple property that a node's key is less than or equal to the node's childrens' keys.	
Graph	A ***graph*** is a data structure for representing connections among items, and consists of vertices connected by edges. A ***vertex*** represents an item in a graph. An ***edge*** represents a connection between two vertices in a graph.	

Choosing Data Structures

- What type of data being stored?
  - Student attributes like last name, first name, UID: Record?
  - Collection of all courses available spring semester: Array? Linked list?
- What type of operations are being performed?
  - Inserting into the middle of a list: Linked list vs. Array

Algorithms

- Sequence of steps to solve a **computational problem**
  - Deterministic
  - Must solve problem
  - Operations a computer can perform
- Problem: mapping of input to output
  - e.g. max: input = a list of unordered integers
    output = the largest value in the list
- Multiple algorithms to solve one problem

Max Problem

- Input:

23	8	108	4	16	42	15

- Algorithm:
  - Start with 1st value, save as currMax
  - Check 2nd value, if larger than currMax, set currMax = currValue
  - Continue until end of list
- OR:
  - Sort list smallest to largest
  - Select last value in list

4	8	15	16	23	42	108

Relation Between DS and Algorithms

- DS define how data is stored…
- and how operations are performed on the DS
- Same operation, different algorithm for different DS
- E.g. append to end

Append to array

1. Increase array's allocation by 1
2. Assign new item as last element
3. Increment array length variable

Append Operation

Array

8	4	16

length = 3

1. Increase array's allocation by 1

8	4	16	

length = 3

2. Assign new item as array's list element

8	4	16	42

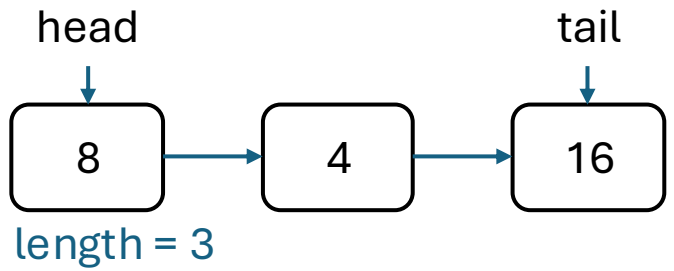
length = 3

3. Increment array length variable

8	4	16	42

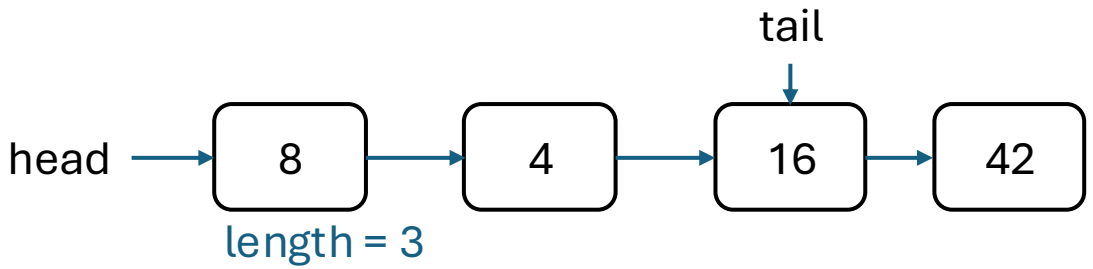
length = 4

Linked List

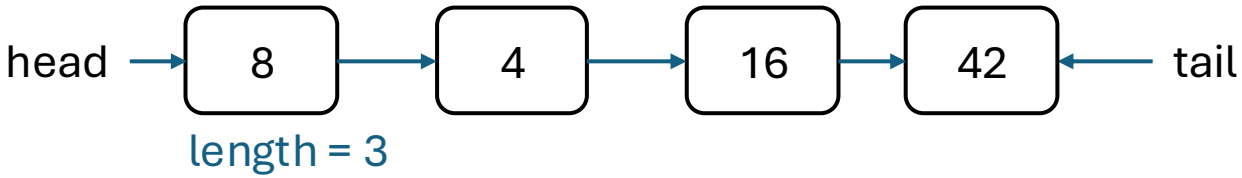


1. Create new node

2. Assign tail's next pointer to new node



3. Assign tail pointer with new node

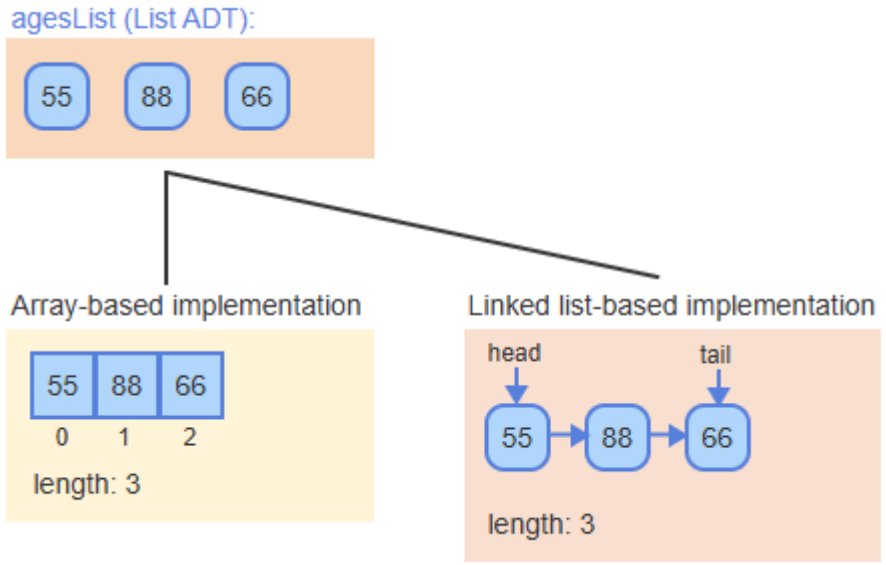


4. Increment list length variable length = 4

What if list is empty?

Abstract Data Types (ADTs)

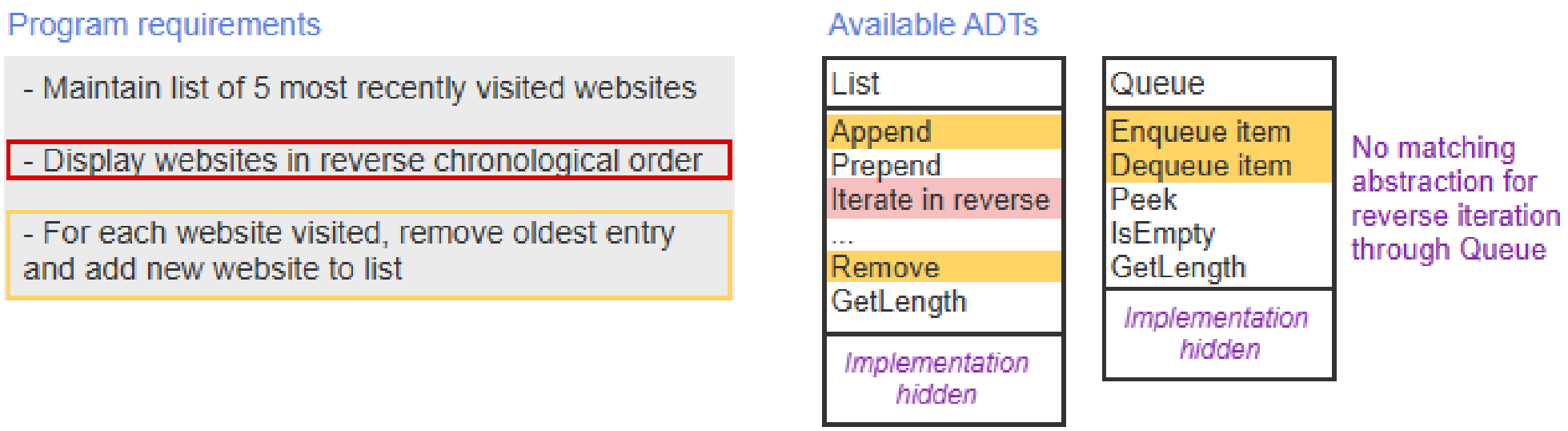
- User operations
  - insert at end
  - remove element
  - get 4th element
- Does not specify how operations implemented
- E.g. List ADT



Abstract data type	Description	Common underlying data structures
List	A ***list*** is an ADT for holding ordered data.	Array, linked list
Dynamic array	A ***dynamic array*** is an ADT for holding ordered data and allowing indexed access.	Array
Stack	A ***stack*** is an ADT in which items are only inserted on or removed from the top of a stack.	Linked list
Queue	A ***queue*** is an ADT in which items are inserted at the end of the queue and removed from the front of the queue.	Linked list
Deque	A ***deque*** (pronounced "deck" and short for double-ended queue) is an ADT in which items can be inserted and removed at both the front and back.	Linked list
Bag	A ***bag*** is an ADT for storing items in which the order does not matter and duplicate items are allowed.	Array, linked list
Set	A ***set*** is an ADT for a collection of distinct items.	Binary search tree, hash table
Priority queue	A ***priority queue*** is a queue where each item has a priority, and items with higher priority are closer to the front of the queue than items with lower priority.	Heap
Dictionary (Map)	A ***dictionary*** is an ADT that associates (or maps) keys with values.	Hash table, binary search tree

Applications of ADTs

- High-level abstraction with internal details hidden
- Focus on higher-level operations and algorithms
- Knowledge of underlying implementation needed to analyze



Algorithm Complexity

- **Algorithm efficiency:** Typically measured by computational complexity
- **Computational complexity:** Amount of recourses used by algorithm
  - Runtime
  - Memory usage

Runtime complexity

- $\boldsymbol{T(N)}$ – number of constant time operations performed with input size N
- Best case – algorithm does minimum number of operations
- Worst case – algorithm does the maximum number of operations
- E.g. Search

23	8	108	4	16	42	15

  - Best case?
  - Worst case?

Runtime complexity

- $T(N)$ – number of constant time operations performed with input size N
- Best case – algorithm does minimum number of operations
- Worst case – algorithm does the maximum number of operations
- E.g. Search

23	8	108	4	16	42	15

  - Best case? search(23)
  - Worst case?

Runtime complexity

- $T(N)$ – number of constant time operations performed with input size N
- Best case – algorithm does minimum number of operations
- Worst case – algorithm does the maximum number of operations
- E.g. Search

23	8	108	4	16	42	15

  - Best case? search(23)
  - Worst case? search (100) (or value not in list)

Space Complexity

- $S(N)$ – number of fixed-size memory units used for input size of n
- E.g. duplicate all number in array:
- = constant, counters, temp pointers, etc.
- Auxiliary space complexity
  - Space used except data (because the data needs space anyway)